



PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE
DEPARTAMENTO DE CIENCIA DE LA COMPUTACIÓN
IIC2283 - DISEÑO Y ANÁLISIS DE ALGORITMOS

Ayudantía 9 - FFT

Octubre de 2025

Caetano Borges, Isabella Cherubini

1. Ejecución FFT

Muestre la ejecución de la FFT sobre los polinomios $P(x) = a_0 + a_1x$ y $Q(x) = b_0 + b_1x$ para transformar el dominio a la representación punto valor, luego multiplique los polinomios en ese dominio, y luego aplique la inversa de la FFT para volver a la representación canónica de los polinomios. A continuación se presenta el pseudocódigo de la FFT:

Algorithm 1 FFT($\bar{a}$)

```
1:  $(a_0, \dots, a_{n-1}) := \bar{a}$ 
2: if  $n = 2$  then
3:    $y_0 := a_0 + a_1$ 
4:    $y_1 := a_0 - a_1$ 
5:   return  $[y_0, y_1]$ 
6: else
7:    $\bar{a}_0 := (a_0, \dots, a_{n-2})$ 
8:    $\bar{a}_1 := (a_1, \dots, a_{n-1})$ 
9:    $[y_{0,0}, \dots, y_{0,\frac{n}{2}-1}] := \text{FFT}(\bar{a}_0)$ 
10:   $[y_{1,0}, \dots, y_{1,\frac{n}{2}-1}] := \text{FFT}(\bar{a}_1)$ 
11:   $\omega_n := e^{\frac{2\pi i}{n}}$ 
12:   $\alpha := 1$ 
13:  for  $k := 0$  to  $\frac{n}{2} - 1$  do
14:     $y_k := y_{0,k} + \alpha \cdot y_{1,k}$ 
15:     $y_{k+\frac{n}{2}} := y_{0,k} - \alpha \cdot y_{1,k}$ 
16:     $\alpha := \alpha \cdot \omega_n$ 
17:  end for
18:  return  $[y_0, \dots, y_{n-1}]$ 
19: end if
```

Solución

Tenemos que $\text{FFT}(\bar{P}) = [a_0 + a_1, a_0 + a_1i, a_0 - a_1, a_0 - a_1i]$, y análogo para $Q(x)$. Luego, el producto de estos dos vectores es

$$\begin{bmatrix} (a_0 + a_1)(b_0 + b_1), \\ (a_0 + a_1i)(b_0 + b_1i), \\ (a_0 - a_1)(b_0 - b_1), \\ (a_0 - a_1i)(b_0 - b_1i) \end{bmatrix}$$

Finalmente, al aplicar la inversa de la FFT obtenemos $[a_0b_0, a_0b_1 + a_1b_0, a_1b_1, 0]$.

2. Aplicación FFT

Sean $A, B \subseteq \{1, \dots, n\}$. Además, sea

$$C = \{a + b \mid a \in A \wedge b \in B\}$$

el conjunto obtenido al sumar cada elemento de A con cada elemento de B . Por ejemplo, si $A = \{1, 2\}$ y $B = \{1, 3\}$, el resultado sería $C = \{2, 3, 4, 5\}$.

Diseñe un algoritmo que genere el conjunto C en tiempo $O(n \log n)$.

Solución

La idea es ver cada conjunto como un polinomio donde cada elemento del conjunto es un exponente del polinomio. Luego al multiplicar los polinomios, los exponentes se sumarán, obteniendo un polinomio con la suma de cada elemento de A con cada elemento de B . Usando el ejemplo del enunciado, creamos $p_A = \sum x^a \mid a \in A = x^1 + x^2$ y $p_B = \sum x^b \mid b \in B = x^1 + x^3$, al hacer multiplicación nos da como resultado el polinomio $p_C = x^2 + x^4 + x^3 + x^5$, el cual pasamos a conjunto y nos da $C = \{2, 4, 3, 5\}$. Luego, cada polinomio lo podemos ver como un arreglo $A[1 \dots n]$ y $B[1 \dots m]$ donde cada posición corresponde a un exponente y su valor corresponde al valor que acompaña a la x , en nuestro ejemplo $A = [0, 1, 1]$ ya que solo tenemos elementos con exponentes 1 y 2 y $B = [0, 1, 0, 1]$ ya que tiene elementos con exponentes 1 y 3. Para lograrlo en la complejidad pedida utilizamos FFT, pero como se vió en clases, necesitamos que los polinomios sean del mismo largo y en particular tamaño exponente de 2, por lo que les agregaremos 0's al final ya que estos no influyen (esto se le llama padding).

Algoritmo SUM-SETS(p_A, p_B)

- 1: $max_len \leftarrow \max\{|p_A|, |p_B|\}$
- 2: $next_exp \leftarrow$ siguiente exponente de 2 después de max_len

```

3:  $pad\_p_A \leftarrow p_A[1, \dots, n, n+1, \dots, next\_exp]$  copiando los elementos de  $p_A$  hasta  $n$ 
   y agregando 0s hasta  $next\_exp$ 
4:  $pad\_p_B \leftarrow p_B[1, \dots, m, m+1, \dots, next\_exp]$  copiando los elementos de  $p_B$  hasta
    $m$  y agregando 0s hasta  $next\_exp$ 
5:  $fft\_p_A \leftarrow FFT(pad\_p_A)$  del doble de largo de  $pad\_p_A$  y cada elemento es una
   tupla (punto, valor)
6:  $fft\_p_B \leftarrow FFT(pad\_p_B)$  del doble de largo de  $pad\_p_B$  y cada elemento es una
   tupla (punto, valor)
7:  $fft\_p_C \leftarrow [(0, 0), \dots, (0, 0)]$  se inicializa con tuplas  $(0, 0)$  y largo  $next\_exp$ .
8: for  $i = 0 \rightarrow next\_exp$  do
9:    $fft\_p_C[i][0] \leftarrow p_A[i][0]$  tomamos el valor del punto de cualquiera de los 2
10:   $fft\_p_C[i][1] \leftarrow p_A[i][1] \cdot p_B[i][1]$  multiplicamos los valores
11: end for
12:  $pad\_p_C \leftarrow IFFT(fft\_p_C)$  aplicamos la inversa de la transformada

```